CMP_SC/INFOTC 3330 - Object-Oriented Programming
Programming Assignment #2
Campus Quest: A Polymorphic Task Board

Spring 2026

Scenario

The university is running a week-long student engagement event called **Campus Quest**. Students earn points by completing different kinds of quests (checking in at events, volunteering, completing a study challenge, etc.). The organizers want a small Java program that can:

- track quests,
- assign quests to students,
- award points when quests are completed,
- and print simple reports.

This assignment is about **good object-oriented design** using **inheritance, interfaces, and polymorphism** to avoid long conditional chains, and using **collections with generics** safely. **Correctness matters, but design quality matters just as much.**

Required Software

Below have hyperlinks, so just click on them to get to the download pages.

- Eclipse IDE (Latest version: 2025-12)**(Required)**
- Java SE Development Kit 25.0.2 (JDK25)

Allowed Topics and Constraints

Stay within what we covered in lecture:

- Inheritance, method overriding, abstract classes
- Interfaces as behavior contracts
- Polymorphism to reduce conditionals and improve extensibility
- Collections Framework (`List`, `Map`, `Set`) and why it exists

- `equals` / `hashCode` basics and their role in hash-based collections
- Generics: generic collections, generic classes, and simple generic methods
- Wildcards (`?`, `extends`, `super`) at a basic level

These topics are explicitly covered in the provided lecture notes.
Do NOT use:

- Java Streams / lambdas
- File I/O, databases, networking, GUIs
- Reflection
- Concurrency
- Any external libraries

Input: No interactive user input is required. Hardcode a demonstration in `Main`.

Submission Instructions (Canvas)

- You must use **Eclipse IDE**.
- Export your Java project as a `.zip` file through Eclipse (do not zip it externally).
- Submit the exported `.zip` on Canvas.
- Your code must compile and run.
- Include a short `README.txt` stating:
  - your name,
  - the parts you have implemented in the project.
- Include screenshots of your output in a folder named `screenshots` (not inside your packages).

Academic Integrity

Use of AI tools is strictly prohibited for this assignment.
Any use of AI generated code, design, or explanations will be reported to the Academic Integrity Office.

What You Are Building

You will build a small quest system with:

- different quest types that award points in different ways,
- a quest board that stores and assigns quests,
- a simple student model with a points total,

- and a few reports.

The core design goal:

> **Avoid long if/else chains that check quest type.**
> Use **polymorphism** so new quest types can be added with minimal changes.

This aligns with the lecture emphasis on replacing conditional-based designs with polymorphic designs.

Required Design Overview

1) `Quest` (interface)

Define a behavior contract for all quests.

Required methods:

- `int getId()`
- `String getTitle()`
- `int getBasePoints()`
- `boolean isCompleted()`
- `int completeFor(Student s)`

Critical rule: `completeFor` must:

- update the quest's completion state,
- award points to the student (directly or indirectly),
- and return the number of points awarded.

Design note: The board should call `quest.completeFor(student)` without needing to know the concrete quest type. This is the point of the interface and polymorphic dispatch.

2) `AbstractQuest` (abstract class)

Create an abstract base class that implements the shared parts of `Quest`:

- common fields (`id`, `title`, `basePoints`, `completed`),
- common validation (fail fast),
- common getters,
- a helpful `toString()`.

Required invariants:

- `id > 0`
- `title` is not null or blank
- `basePoints > 0`

Required behavior:

- A constructor that validates invariants (throw `IllegalArgumentException`)

Abstract classes and abstract methods are part of the lecture content.

3) Concrete quest types (at least 3 classes)

Implement at least **three** different quest types, each with different completion behavior. You must create these three, and you may add more.

A. `EventCheckInQuest`

A student completes this by checking in at an event.

Required additional data:

- `String eventName` (not null/blank)

Completion rule:

- awards exactly `basePoints`

B. `VolunteerQuest`

A student completes this by volunteering some number of hours.

Required additional data:

- `int hours` (must be positive)

Completion rule (you must implement):

- awarded points = `basePoints * hours`

C. `StreakQuest`

This encourages consistent behavior.

Required additional data:

- `int days` (must be positive)

Completion rule (open design choice): Choose ONE of the following approaches and document it in a short comment near the formula:

- linear bonus: `basePoints + (days * 2)`
- milestone bonus: extra bonus if `days >= 5`
- your own clear, reasonable rule

Your choice must be consistent and validated.

Important: Do not implement these differences using conditionals in the quest board. Put the behavior inside the quest classes (polymorphism).

4) `Student`

Represents a participating student.

Required data:

- `String name` (not null/blank)
- `int points` (starts at 0, never negative)

Required behavior:

- `addPoints(int amount)` with validation (fail fast)
- `toString()` includes name and points

Required `equals` and `hashCode`: Students will be stored in a `HashSet<Student>`. Implement `equals` and `hashCode` so that two `Student` objects are considered equal if their `name` matches exactly. This is directly tied to how hash-based collections work.

5) `QuestBoard`

This class coordinates storage and assignment.

Required fields (minimum):

- `Map<Integer, Quest> questsById`
- `Map<Student, List<Quest>> assignments`

Required behavior:

- `addQuest(Quest q)`
  - rejects duplicate quest IDs (throw `IllegalArgumentException`)
- `Quest findQuest(int id)` returns quest or `null`
- `assignQuest(Student s, int questId)`
  - assigns an existing quest to a student
  - open design choice: decide if the same quest can be assigned multiple times to the same student
- `int completeQuest(Student s, int questId)`
  - finds the quest in that student's assignments
  - calls `quest.completeFor(s)` polymorphically
  - returns awarded points
- `void printAllQuests()`
- `void printAssignmentsFor(Student s)`

Design requirements:

- Keep fields private.
- Do not expose internal collections directly.
- Avoid long chains and knowledge leaks.
- The board should not do type-checking like `if (q instanceof VolunteerQuest)`.

6) `RewardUtil` (generic utilities)

Create a small utility class to practice generics safely.

Requirement A: Generic method

- Implement `public static <T> void printAll(List<T> items)`
- It prints each item on its own line using `toString()`.

Generic methods are part of the lecture notes. :contentReference[oaicite:8]index=8

Requirement B: Wildcard method

- Implement `public static int sumPoints(List<? extends Student> students)`

It returns the total points across the list.

This uses an upper bounded wildcard to safely read from a list of students (or subclasses, if you ever had them). Wildcards and bounds are part of the lecture notes.

Main Demo Requirements

Your `Main` must be a hardcoded demo that shows the system working.

Minimum demo:

- Create at least 3 students, including one intentional duplicate name to demonstrate `HashSet<Student>` behavior.
- Create at least 6 quests total (at least 2 of each required quest type).
- Add quests to the board.
- Assign at least 4 quests across at least 2 students.
- Complete at least 3 quests across different quest types.
- Print:
  - all quests
  - assignments for each student
  - final student point totals
  - the total points across students using `RewardUtil.sumPoints`

Fail fast requirement: demonstrate at least **one** invalid action and show that your program throws an exception (example: adding a quest with duplicate ID, or completing a quest twice). Do not silently ignore errors.

Open Design Choices (You Must Decide and Document)

You must choose rules and enforce them consistently. Add a short comment where the rule is enforced.

0) Can a quest be completed more than once?

- Recommended: no, completing twice should throw `IllegalStateException`.

0) Can the same quest ID be assigned to the same student multiple times?
   - If no: enforce using logic in `QuestBoard.assignQuest`.
   - If yes: document how completion works (which instance completes?).

0) For `StreakQuest`, define a clear scoring rule and validate its data.

Deliverables

Submit your exported project as a .zip file through the Eclipse IDE. Your project must have the files below at minimum (you may include more if your design is clean and justified):

- `Quest.java`
- `AbstractQuest.java`
- `EventCheckInQuest.java`
- `VolunteerQuest.java`
- `StreakQuest.java`
- `Student.java`
- `QuestBoard.java`
- `RewardUtil.java`
- `Main.java`
- `README.txt`
- Have your screenshots in your project under a folder called "screenshots", the folder shouldn't be under your source directory.

Grading Rubric (40 points total)

1. Correctness and Required Features (14 points)

- Program compiles and runs; required files included; demo is complete (3 points)
- Quest types behave correctly and award the correct points (4 points)
- QuestBoard supports add, find, assign, complete, and printing as specified (5 points)
- Fail fast behavior demonstrated correctly (exceptions thrown for invalid actions) (2 points)

2. Polymorphism and OO Design (10 points)

- Uses `Quest` interface and polymorphic calls properly (no type-checking conditionals in the board) (4 points)
- Appropriate use of an abstract base class for shared quest behavior (2 points)
- Responsibilities are well placed (no god class, no anemic design) (2 points)
- Avoids long message chains and knowledge leaks (Law of Demeter style) (2 points)

3. Collections, `equals`/`hashCode`, and Data Integrity (8 points)

- Correct use of generic collections (`Map`, `List`, `Set`) with type parameters (3 points)
- `Student.equals` and `Student.hashCode` are consistent and enable correct `HashSet` behavior (3 points)
- Duplicate quest ID detection and other integrity rules are enforced reliably (2 points)

4. Generics Utilities (5 points)

- `RewardUtil.printAll(List<T>)` implemented and used in the demo (2 points)
- `RewardUtil.sumPoints(List<? extends Student>)` implemented correctly and used in the demo (3 points)

5. Readability, Documentation, and Submission Quality (3 points)

- Clear naming, consistent formatting, meaningful `toString` output (2 points)
- README, screenshots, and Canvas submission format are correct (Eclipse-exported zip) (1 point)

Notes to Students

There are multiple valid designs. You will be graded on:

- how well your design uses polymorphism to avoid conditionals,
- how well your classes protect their invariants (fail fast),
- and how consistently you enforce your own documented rules.